Task 3: Unions and Memory Layout (9 Marks)

Task 3.1: Structure or Union? (3 Marks)

Design A: structure holding all three fields

```
struct packet_all {
    enum kind type;
    float celsius;
    int   percent;
    char  status[8];      /* up to 7 characters + '\0' */
};
```

Design B: tagged union

```
enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;               /* tag: tells which member is valid */
    union {
        float celsius;
        int   percent;
        char  status[8];
    } value;
};
```

Justification. A packet carries exactly one kind of value, so the three fields are never needed at the same time. Design A reserves space for all three in every packet, so two of the three fields are always wasted. Design B overlays them, so each packet only needs room for the largest member plus a tag. With thousands of packets and very little memory, Design B is the right choice. At 10,000 packets it saves 80,000 bytes (about 78 KB, a 40% saving). Its cost is that the programmer must always check the tag before reading a member, which is why the union is never used without the tag.

Task 3.2: Predict the Size (2 Marks)

Design	Layout (typical 64-bit GCC)	Predicted / measured
Union alone	Members: float 4, int 4, char[8] 8. All start at offset 0 and overlap. Size = largest member = 8. Alignment = 4 (strictest member is float/int).	8 / 8
Design A (all fields)	type 0-3, celsius 4-7, percent 8-11, status 12-19. Total 20, which is already a multiple of the alignment 4, so no padding.	20 / 20
Design B (tagged union)	type 0-3, union starts at offset 4 (a multiple of its alignment 4), union 4-11. Total 12, multiple of 4, so no padding.	12 / 12

Alignment and padding. Each member must start at an address that is a multiple of its alignment, and the structure size must be a multiple of the strictest member alignment. Here every member needs at most 4-byte alignment, so no padding bytes are inserted. If the union contained a `double` (alignment 8), the compiler would insert 4 padding bytes after the tag and the structure would grow to 16 bytes. The values 8, 20 and 12 were confirmed with `sizeof`.

Task 3.3: Implement and Test (3 Marks)

The complete program, including `show()` (the answer to TODO 1) and the misuse experiment of Task 3.4:

```
#include <stdio.h>
#include <string.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;
```

```
    union { float celsius; int percent; char status[8]; } value;
};

struct reading_all {          /* "all three fields" design */
    enum kind type;
    float celsius; int percent; char status[8];
};

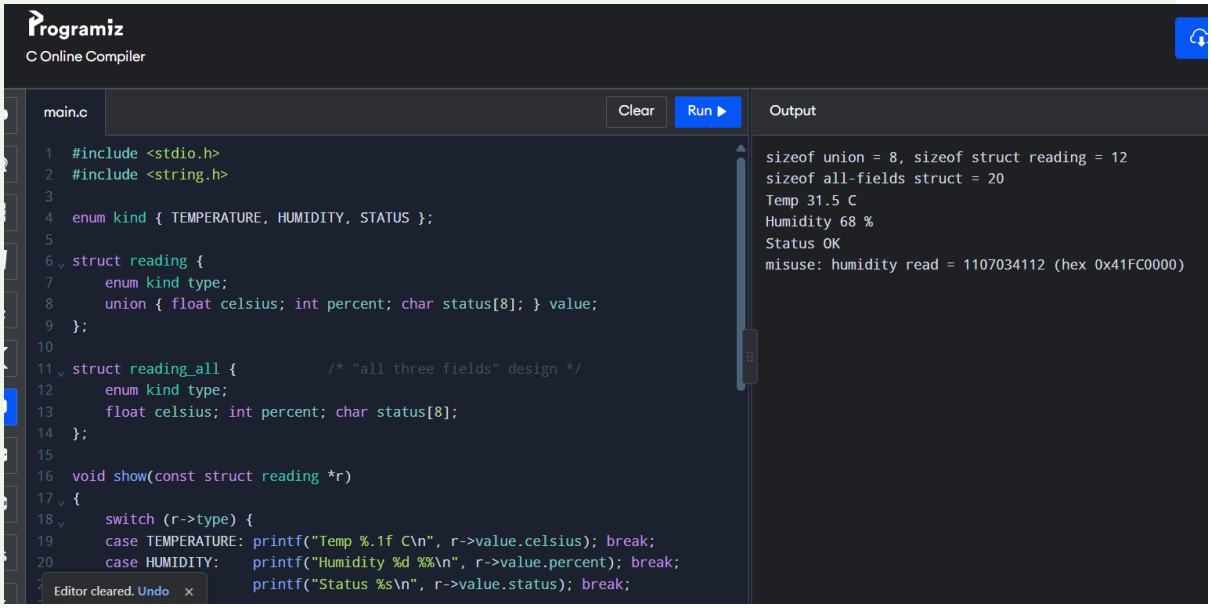
void show(const struct reading *r)
{
    switch (r->type) {
    case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
    case HUMIDITY:    printf("Humidity %d %%\n", r->value.percent); break;
    case STATUS:      printf("Status %s\n", r->value.status); break;
    }
}

int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    printf("sizeof all-fields struct = %zu\n", sizeof(struct reading_all));
    for (int i = 0; i < 3; i++) show(&log[i]);

    /* misuse experiment */
    struct reading m; m.type = TEMPERATURE; m.value.celsius = 31.5f;
    printf("misuse: humidity read = %d (hex 0x%X)\n", m.value.percent, (unsigned)m.value.percent);
    return 0;
}
```

Output

```
sizeof union = 8, sizeof struct reading = 12
sizeof all-fields struct = 20
Temp 31.5 C
Humidity 68 %
Status OK
misuse: humidity read = 1107034112 (hex 0x41FC0000)
```



Why 12 bytes and not 4 + 4 + 4 + 8? The sum 4 + 4 + 4 + 8 = 20 would be the size if the three values were separate fields (that is Design A). In a union, all members share the same memory starting at one

address, so the union occupies only the size of its largest member (8 bytes), not 4 + 4 + 8. The structure is then the 4-byte tag plus the 8-byte union = 12 bytes.

Task 3.4: Misuse Experiment (1 Mark)

A temperature of 31.5 is stored in value.celsius, then value.percent is read. The output is **1107034112** (hexadecimal 0x41FC0000).

Explanation. celsius and percent share the same 4 bytes. 31.5 is stored in IEEE-754 single-precision format: sign 0, exponent 10000011 (that is 127 + 4), fraction 11111000...0, which makes the bit pattern 0x41FC0000. Reading the same bits as an int does no conversion; it merely reinterprets the pattern, giving 1107034112. The number is meaningless as a humidity. This is the reason for the type tag: the program must read only the member that was last written.

Alternative Concept and Complexity Analysis

Alternative 1: three separate arrays (float temp[N], int hum[N], char status[N][8]). **Alternative 2: each packet allocated with malloc** and referenced through a void * (a tag is still needed, because a void * carries no type).

	Tagged union array	**Three separate arrays**	**malloc + void ***
Time to read packet i	O(1): one indexed load, contiguous memory	O(1) if the type and index are known; the arrival order across the three types is lost unless an extra (type, index) table is kept	O(1) but needs an extra pointer dereference, a likely cache miss, and a cast; storing needs malloc (slower, not constant-time guaranteed)
Space per packet	12 bytes	4 / 4 / 8 bytes with no tag, but each array must be sized for the worst case, so unused capacity is wasted; plus the (type, index) table (about 8 bytes per packet) to keep order	Pointer 8 + tag 4 (+4 padding) = 16 bytes in the array, plus the heap block (typically 24 bytes minimum on glibc, with 8 to 16 bytes of malloc bookkeeping): about 40 bytes
Other issues	Compact; the tag must be respected	Awkward to process packets in time order	Memory leaks, fragmentation, type errors from wrong casts, free needed for every packet

For a station with very little memory that stores thousands of packets, the tagged union is the best design: smallest per packet, constant-time access and no allocation.